

Flask & RESTful APIs

From Basic to Advanced – A Project Development Perspective

Dr. Anilkumar V. Brahmane

Computer Engineering Department

Sanjivani College of Engineering

- ❏ Flask helps us create a web server/API in Python, allowing other applications to communicate with our Python app – including ML and AI models – over the internet.





What is an API? The Restaurant Analogy

Real-Life Analogy

In a restaurant, the **customer** never enters the kitchen directly. The **waiter** acts as the interface – taking requests and delivering responses.

Customer → Waiter → Kitchen → Food
→ Customer

In Software

Similarly, the **frontend** never directly touches the backend. The **API** acts as the communication interface.

Frontend → API → Backend → Database
/ ML Model

📄 API = Application Programming Interface – a messenger between two software applications.

— Dr. Anilkumar V. Brahmane | Computer Engineering Dept., Sanjivani College of Engineering

Why Flask for ML Projects?

Flask is a **lightweight Python web framework** for building web apps, REST APIs, and ML/AI backends. Without Flask, exposing a Python ML model to the web requires much more low-level work.

The Problem

After training a model with `model.fit(X_train, y_train)`, your website, Android app, or IoT device **cannot call a Python variable directly**.

The Solution

Wrap the model in a **Flask REST API**. Now any application — web, mobile, IoT — can send data and receive predictions via HTTP.

Project Value

Students can take any ML/AI model and **expose it as an API** that a web or mobile frontend can consume — a core industry skill.

— Dr. Anilkumar V. Brahmane | Computer Engineering Dept., Sanjivani College of Engineering

Your First Flask Application

Install & Run

```
pip install flask
```

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, World!"

if __name__ == "__main__":
    app.run(debug=True)
```

Run with `python app.py`. Visit `http://127.0.0.1:5000` to see **Hello, World!**

Understanding Routes

A **route** tells Flask: "When someone requests this URL, execute this function."

```
@app.route("/students")
def students():
    return "Student Page"
```

You can define **multiple routes** for `/`, `/about`, `/students`, etc. — each mapped to its own function.

❏ Routes are the building blocks of every Flask project endpoint.

HTTP Methods, JSON & REST

The Four HTTP Methods (CRUD)

Method	CRUD	Purpose
GET	Read	Fetch data
POST	Create	Send/create data
PUT	Update	Modify existing data
DELETE	Delete	Remove data

REST Resource Design

```
GET /students # All students
POST /students # Add student
GET /students/101 # Student 101
PUT /students/101 # Update 101
DELETE /students/101 # Delete 101
```

JSON — The Common Language

JSON (JavaScript Object Notation) is the standard data format for REST APIs. React, Flask, and Python all speak JSON.

```
{
  "name": "Rahul",
  "age": 20,
  "branch": "CE"
}
```

Flask POST Example

```
data = request.get_json()
name = data["name"]
return jsonify({
  "message": "Student added",
  "name": name
}), 201
```

Flask + ML Model: The /predict API

This is the most important pattern for AI/ML student projects. Save your trained model, load it in Flask, and expose a `/predict` endpoint.

Save & Load Model

```
import joblib
# Save
joblib.dump(model, "student_model.pkl")
# Load in Flask
model = joblib.load("student_model.pkl")
```

The /predict Endpoint

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    study_hours = data["study_hours"]
    attendance = data["attendance"]
    prev_marks = data["previous_marks"]
    pred = model.predict(
        [[study_hours, attendance, prev_marks]]
    )
    result = "PASS" if pred[0]==1 else "FAIL"
    return jsonify({"prediction": result})
```

How It Works



Client sends `{"study_hours":6, "attendance":85, "previous_marks":70}` and receives `{"prediction":"PASS"}`.

Advanced Flask Concepts for Projects

URL & Query Parameters

Use `/students/<int:student_id>` for path params, or `/students?branch=CE&year=4` with `request.args.get()` for query params.

HTTP Status Codes

Return 200 (Success), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 404 (Not Found), 500 (Server Error) for professional APIs.

Input Validation & Error Handling

Always validate inputs before passing to the model. If `study_hours` is missing or `-50`, return a 400 error with a meaningful message – never crash silently.

CORS for React Integration

Install `flask-cors` and use `CORS(app)` so your React frontend at `localhost:3000` can call your Flask API at `localhost:5000` without browser blocks.

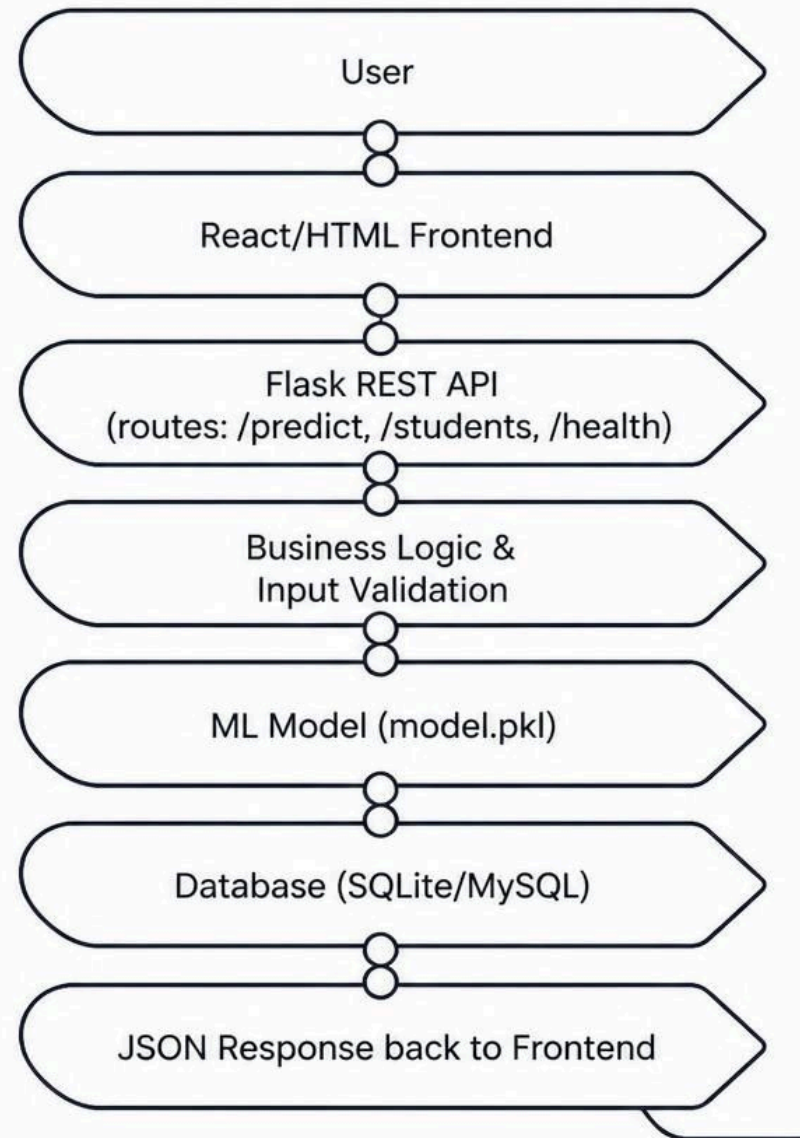
API Authentication

For private ML APIs, implement API keys or JWT tokens. Students can demonstrate the concept with a simple API-key check; production systems require stronger security.

API Versioning

Use `/api/v1/predict` so existing apps aren't broken when you release `/api/v2/predict` – a critical industry practice.

Complete Project Architecture



This architecture separates concerns: the frontend handles UI, Flask handles routing and validation, the ML model handles inference, and the database stores predictions. Each component has a **single responsibility** — far better than putting everything in one `main.py`.

Student Project Ideas Using Flask + ML



Student Performance Prediction

Input: study hours, attendance, marks → Flask → ML Model → Pass/Fail



House Price Prediction

Input: house details → Flask API → Regression Model → Predicted Price



Disease Risk Prediction

Patient parameters → Flask → ML Model → Risk score (research/decision-support prototype only)



Flask + LLM / RAG Chatbot

User question → Flask API → RAG + Vector DB + LLM → Answer. Ideal for Agentic AI projects using Gemini or OpenAI-compatible APIs.

Other ideas: Loan Approval, Fake News Classification, Sentiment Analysis, Image Classification, Crop Disease Detection (CNN).

— Dr. Anilkumar V. Brahmane / Computer Engineering Dept., Sanjivani College of Engineering

Learning Roadmap & Key Terms

Three-Level Learning Path

01

Level 1 — Flask Basics

Installation, routes, GET/POST, JSON, request, jsonify, status codes

02

Level 2 — ML Integration

Train → Serialize → Load → /predict API → Input validation → Error handling

03

Level 3 — Industry Project

React frontend + Flask + DB + Auth + CORS + Versioning + Postman + Docker + Cloud + Monitoring

10 Terms Every Student Must Know

Term	Meaning
Flask	Python web framework
API	Communication interface
REST	Web API design style
Route	URL handled by app
GET / POST	Read / Send data
JSON	Common data format
Endpoint	Specific API URL
Serialization	Save trained model
Inference	Model makes prediction

📌 Practical sequence: Scikit-learn → joblib → Flask → REST API → /predict → Postman → React → Full ML Web App → Deployment